Abdelrahman Rashed 900231248, Ziad Khairaldin 900225292, DD2 (CSCE 3304) Fall 2026, group gb2.

DD2 Project 1, Option 2 - Finding Cells Worth Adding

We chose Option 2 as a three-student group, which means we also had to build real cells with the course's cell generator. We used AI only to help us write the code. The repository is github.com/Gad101/cellgap, under the tag final. The code and results are at commit 1b5b63c; the tag only adds this report on top of that commit. Every number in this report was written by our tool into a file in the results/ folder, and we name that file next to the number so it can be checked.

part 1: introduction

The problem, and what we built.

The HD standard-cell library (sky130_fd_sc_hd) already contains most of the small functions that fit in a single CMOS stage. The question this project asks is: which small logic functions do real designs compute over and over, does HD lack a cell for any of them, and if we add such a cell, do the designs actually get smaller when Yosys maps them to the library?

To answer this we built a tool called cellgap. It does the 12 steps listed in section 4 of the handout. It is deterministic, meaning it gives the same output every time for the same input, and no AI model runs inside it. The commands are:

- make all runs steps 1 to 9.
- make final runs steps 10 and 12 and produces the final tables.
- make real-cells and make real-lib run step 11.
- make demo IN=<design> TOP=<top module> runs the whole flow on a single design. This is the run we will show at the interview.

The result in short: our tool estimated that the new cells would make the required designs 0.17% smaller in total. When we replaced the estimated cells with the real cells built by the generator, the saving dropped to 0.01%. Both figures are measured, and every mapped design was checked to be logically equivalent to the original. Part 5 explains why the saving dropped.

part 2 step 1:

Turn each design into an and-inverter graph, before and after ABC restructuring.

An and-inverter graph (AIG) describes a circuit using only 2-input AND gates and inverters. The script src/make_aig.sh builds it with Yosys. It reads the Verilog, flattens the design into one module (synth -flatten), converts asynchronous resets into synchronous ones (async2sync), turns the flip-flops into plain logic with registers (dffunmap), rewrites all logic as AND gates and inverters (aigmap), and writes the graph to a file (write_aiger -zinit). The -zinit option makes every register start at zero. With this option our AND-gate counts match Table 2 of the handout exactly for all six required designs.

We also make a second version of each graph after ABC has restructured it. The script src/abc_after.sh runs the ABC commands strash, dch and strash again. The final strash is needed because dch leaves behind extra "choice" nodes that are not connected to anything, and strash

removes them. We then use ABC's cec command (combinational equivalence check) to prove that the restructured graph computes exactly the same function as the original.

part 2 step 2:

Find every cut of up to 4 inputs, compute its truth table, and check it by simulation.

A cut is a small piece of the graph: one node (the root) together with up to 4 nodes (the leaves) that completely determine its value. Each cut therefore computes a function of at most 4 inputs, which can be written as a 16-bit truth table.

We wrote our own cut finder, src/cuts.py. It works from the inputs of the graph up to the outputs and does not limit how many cuts it keeps per node, so no cut is missed. To make sure every truth table is right, the tool simulates each cut again for all 16 combinations of its leaf values and compares the result with the stored table. It also checks that the leaves really separate the root from the rest of the circuit, so that the cut is valid. Both checks pass on all 16 graphs (8 designs, each before and after ABC).

part 2 step 3:

Group the functions into classes.

Many different truth tables are really the same function with the inputs in a different order, or with the output inverted. We group such functions into one class, because one cell can implement all of them.

The file src/tt.py precomputes the class of every one of the 65,536 possible 4-input truth tables, under three rules:

- P: functions are the same if one is the other with the inputs reordered.
- PO: the same as P, but an inverted output also counts as the same function.
- NPN: the same as PO, but inverted inputs also count as the same function.

PO is the class we actually cost and compare, because in CMOS reordering inputs and inverting the output are free, while inverting an input is not. We count NPN classes only for comparison (part 5).

part 2 step 4:

Work out what each class would cost as a CMOS cell.

For every class, src/costmodel.py and src/cmos.py find the cheapest way to build it in CMOS, with no more than 3 transistors in series. They try three options and keep the cheapest:

1. one static CMOS stage;
2. one stage with inverters added on some inputs or on the output;
3. splitting the function into several stages (a Shannon split, which breaks the function on one input).

The cost is given as the number of stages, the number of inverters and the number of transistors. Each cost also comes with a Boolean formula describing the circuit. The tool simulates that formula and requires it to produce the class it claims to build. All 3,984 formulas pass this check.

part 2 step 5:

Check each class against HD and drop the ones HD already has.

The script src/redundancy.py compares every class with every single-output combinational cell in the full HD Liberty file, at every drive strength. Following section 5 of the handout, each class gets one of four labels:

- Exists: HD already has a cell for it.
- Exists with an inverter: HD has a cell for it if an inverter is added.
- Contained: HD has a bigger cell that can compute it, for example by tying some inputs.
- New: HD has nothing for it.

We also apply the area exceptions from Table 3: a class that Exists with an inverter, or is Contained, is still kept if our estimated cell would be smaller than the HD solution. As a test of this checker, it reproduces the handout's Table 4 (part 3).

part 2 step 6:

Estimate the area of a new cell, using a model fitted to HD's x1 cells.

Before a cell is built we cannot know its exact area, so we estimate it. We fitted a straight line of cell width (in placement sites) against the number of transistors, using HD's x1 cells. We fitted one line for single-stage cells and another for multi-stage cells (results/area_model.json):

- Single-stage cells: sites = 1.053 + 0.553 x (transistor count). It is fitted to 13 cells. On average it is off by 0.62 sites (15.9%), and at worst by 1 site (a21oi_1).
- Multi-stage cells: sites = 2.512 + 0.461 x (transistor count). It is fitted to 45 cells. On average it is off by 0.98 sites (14.9%), and at worst by 3 sites (or4_1).

We left nand4_1 and nor4_1 out of the fit because they have 4 transistors in series, which our cells are not allowed to have.

part 2 step 7:

Rank the candidate cells by how much area they are expected to save.

The script src/candidates.py estimates the saving for each candidate class. One place where the function occurs saves:

(the number of AND gates that only this function uses) x (the design's average area per AND gate) - (the estimated area of the new cell).

The number of AND gates is limited to the smallest count seen for that function, so that one unusually large occurrence does not inflate the estimate. Within each design the tool only counts occurrences that do not overlap, and then ranks the classes by their total saving over all designs. As the handout says, this is only a guess. Steps 9 and 10 measure the real saving.

part 2 step 8:

Write a cell request for each candidate, and a Liberty file containing the new cells.

The script src/write_cell_requests.py writes one request per candidate in the format of Appendix A. It refuses to write a request if the Boolean expression and the truth table do not agree. The script src/write_liberty_with_new_cells.py takes hd_reduced.lib and adds the new cells to it in the format of section 6. To check the result, Yosys must load exactly 189 + N cells (189 from HD plus N new ones), and the function of every new cell must equal its request.

part 2 step 9:

Map every design with and without the new cells, and check that the result is correct.

We map each design with the commands from section 2 of the handout, using both mapping scripts, A and B. The only thing that changes between runs is the library. Without new cells our areas match Table 1 exactly.

Every mapped netlist is checked against the original RTL with ABC: cec for designs without flip-flops, and dsec (sequential equivalence check) for designs with flip-flops. For the SPM and mul designs we also run the gate-level testbenches from Tutorial 4 in Icarus Verilog.

part 2 step 10:

Choose the final set of cells, at most 10, one at a time.

The script src/select_final_set.py goes down the ranked list. For each candidate it adds the cell to the library, maps the six required designs again, and keeps the cell only if the total area goes down by more than 0.05 um2. The total is taken as the better of script A and script B for each design. It stops after 10 kept cells or at the end of the list.

part 2 step 11:

Build the final cells with the cell generator and use the real cells.

The script src/run_cellweaver.py runs the cellweaver generator on each final request. For each cell it runs the Magic design-rule check (DRC), the Netgen layout-versus-schematic check (LVS), a check that the cell can sit next to other cells (abutment), timing characterization from the extracted layout, and a check of the resulting Liberty file in OpenSTA. Then src/real_lib.py replaces each estimated cell in our Liberty file with the real one and runs the loading and function checks again. The folder cells/<cell>/ holds each cell's GDS, LEF, Liberty and Verilog files, and the Appendix A response.

part 2 step 12:

Make one new cell 30% smaller and map again.

The script src/sensitivity.py takes the final cell that is used most often, reduces its area in the Liberty file by 30%, keeps everything else the same, and maps all designs again. This shows how much the result depends on our area estimate.

part 2: worst delay

Report the worst delay of each design from OpenSTA.

The script src/worst_delay.py runs OpenSTA with the same settings for every netlist: a 10 ns clock on the port that drives the flip-flops, all other inputs arriving at 0 ns, and no output load. The worst delay is the latest time at which data arrives anywhere in the design. We leave out the reset recovery paths, since they are not part of the normal data path.

part 3: verification

Every check we run, and a deliberately broken case that proves each check works.

The command make test runs 63 tests inside the course's Docker image. Two more tests need Icarus Verilog and run on the host computer. A check only counts if we have shown that it fails on a broken input, so each check below has one.

The request's expression must match its truth table. The broken file tests/broken/wrong_expression.json gives truth table 0x077E with the formula for 0x077F. The tool refuses it.

Each cut's truth table must match simulation. wrong_cut_table.json has one row of a truth table flipped. The check catches it.

The leaves must really cut off the root. not_a_cut.json has one leaf removed. The check catches it.

The redundancy checker must be correct. All 258 HD cells with up to 4 inputs are labelled Exists, and Table 4 is reproduced. When we use hd_missing_cells.json, which is HD with the a22oi cell removed, Table 4 no longer matches, so the checker notices a missing cell.

Each cost formula must compute its own class. A formula for a different function (0x0707 written with !(A&B)) is rejected.

The graph after ABC must equal the original. A copy of csa8 with one AND-gate input inverted is rejected by cec.

The Liberty file must load, with 189 + N cells and correct functions. A file with an unclosed quote is rejected. A file with a missing brace is still read by Yosys 0.33, but only 190 cells are loaded, so the cell-count check fails. A cell whose function names a pin that does not exist is reported.

Every mapped netlist must equal the RTL. In mul (checked with cec) and in cpu8 (checked with dsec) we swapped one nand2_1 for a nor2_1. Both are rejected.

The gate-level testbenches must catch errors. The same swap makes mul_exhaustive_tb fail.

The greedy selection rule. A cell that makes the total larger is dropped, and so is a cell whose gain is smaller than rounding error.

The final table. A failed equivalence check shows up as FAIL in the table. A real Liberty file with a missing or unbalanced cell group is refused.

Calibration against the handout.

- Table 1 is reproduced exactly: all 24 areas differ by 0.000% (results/table1.md, using native Yosys 0.33).
- Our AND-gate counts equal Table 2.
- Table 4 is reproduced: 24 Exists, 20 Exists with an inverter, 38 Contained and 3,822 New; 9 New single-stage classes; and 8 read-once single-stage classes, of which 6 Exist and 2 are Contained. One row differs: we count 6 where the handout has 5 (explained in part 6).

part 4 step 1: designs, cuts and classes

The size of each design's graph, its number of cuts and its number of classes, before and after ABC (results/design_stats.csv).

Each entry shows the value before ABC / after ABC.

Design	AND gates	Cuts	P classes	NPN classes
cpu8	1,265 / 1,109	10,725 / 9,007	281 / 233	63 / 59
dsp8	1,973 / 1,532	16,445 / 13,032	268 / 252	54 / 50
arlet6502	3,221 / 2,699	27,091 / 20,438	312 / 287	51 / 60
uart_apb	4,673 / 2,971	39,290 / 17,934	319 / 180	45 / 38
risc8	7,750 / 6,592	57,178 / 44,144	276 / 223	45 / 47
fir10	8,795 / 6,937	92,196 / 75,435	232 / 212	43 / 43
spm	953 / 540	10,563 / 3,179	109 / 51	29 / 22
mul	774 / 512	8,221 / 5,356	242 / 196	52 / 36

Across all eight designs we found 362 PO classes before ABC (316 after) and 91 NPN classes (86 after) (results/class_counts.csv). For comparison, HD's own 258 small cells cover only 29 PO classes (results/hd_class_table.md).

part 4 step 2: redundancy report

Each candidate class, with its Table 3 label and the HD cell it matches (results/redundancy_report.csv).

In total 556 classes appear in the cuts of the real designs. The table shows how many got each label, and how many we kept as candidates.

Table 3 label	Classes found	Kept as candidates
Exists	53	0
Exists with an inverter	28	23, because our estimated cell is smaller than the HD cell plus an inv_1
Contained	30	28, because our estimated cell is smaller than the HD cell
New	445	445

Some examples:

- 0x000F is a 2-input NOR, so it Exists (nor2).

- 0x00EE is HD's o21ba cell, so it Exists.
- 0x07FF, which is !(D & (C | A&B)), is Contained in a32oi. We kept it because our estimated cell is 7.51 um2 while a32oi is 8.76 um2.
- 0xEEFA is Contained in mux4. We kept it because our estimated cell is 12.51 um2 while mux4 is 22.52 um2.

part 4 step 3: ranking and final set

The ranked candidates with their estimated savings, and how the final set was chosen (results/candidates.json, results/greedy_log.csv, results/final_set.json).

Only 11 candidates have a positive estimated saving. For each one, the greedy step added it, mapped the six required designs again, and looked at the new total area. "T / stages" means number of transistors / number of CMOS stages.

Try	Class	T / stages	Estimated area (um2)	Estimated saving (um2)	Total area after (um2)	Decision
1	0xEEFA	16 / 3	12.51	36.74	86,282.8	dropped (total grew by 37.5)
2	0x8AE6	34 / 4	23.77	25.92	86,245.2	dropped
3	0x17	10 / 1	8.76	12.01	86,235.2	**kept**
4	0x077F (the handout's maj4n)	12 / 1	10.01	8.25	86,235.2	dropped
5	0x17FF	12 / 1	10.01	5.50	86,234.0	**kept**
6	0xBCC4	34 / 4	23.77	3.70	86,234.0	dropped
7	0x07FF	8 / 1	7.51	2.84	86,102.6	**kept**
8-11	0xAABF, 0x0BBF, 0x2BFF, 0x0EEF			1.59 down to 0.74		dropped

The final set is three cells, all single-stage: dd2_gb2__f17_1, dd2_gb2__f17ff_1 and dd2_gb2__f07ff_1.

part 4 step 4: Table 5 with estimated areas

The original library compared with the library plus our cells, for scripts A and B and for the better of the two (results/table5_final_est.md; that file also has the rows for each script, the instance counts and the delays).

All 32 mappings pass the equivalence check (cec or dsec). Taking the better of script A and script B for each design, the area went down by:

Design	Area reduction
dsp8	0.23%
arlet6502	0.49%
uart_apb	0.07%
risc8	0.26%

Design	Area reduction
cpu8, fir10, spm, mul	0.00%
Total of the required designs	**86,245.3 -> 86,102.6 um2 (0.17%)**

If we compare script B with script B only, the savings are larger: fir10 2.10% (using 241 instances of the new cells), cpu8 1.75%, mul 1.41% and arlet6502 1.11%. Script A only ever uses the f07ff cell.

part 4 step 5: real cells

The final cells with their real areas from the generator (results/real_cells/, cells/, results/real_vs_estimated_area.csv).

cellweaver built all three cells at the standard single row height. All of them pass DRC, LVS, the abutment check in all four orientations, and timing characterization from the extracted layout, and OpenSTA accepts each Liberty file. The real cells are much larger than we estimated:

Cell	Estimated area (um2)	Real area (um2)	Real divided by estimated
f17	8.76	17.52	2.0x
f17ff	10.01	30.03	3.0x
f07ff	7.51	16.27	2.17x

part 4 step 6: Table 5 with real cells

The same table, using the real cells (results/table5_final_real.md).

All 32 mappings pass the equivalence check. With the real areas, ABC almost never chooses our cells: fir10 with script B uses 2 of them, and no other mapping uses any. The total area, taking the better script for each design, goes from **86,245.3 to 86,239.0 um2, a saving of 0.01%**. The only design that gets smaller is risc8, by 0.02%.

part 4 step 7: cell-by-cell curve

The total area after each final cell is added, in the order the tool chose them (results/curve.md, results/curve_real.md, and the SVG plots next to them).

Cells added	Total with estimated areas (um2)	Total with real areas (um2)
0	86,245.2	86,245.2
1 (f17)	86,235.2	86,246.5
2 (f17 and f17ff)	86,234.0	86,244.0
3 (all three)	86,102.6	86,239.0

part 4 step 8: the 30% experiment

One new cell made 30% smaller in the Liberty file, and the designs mapped again (results/sensitivity_30pct.md).

We made f17 smaller, from 8.76 to 6.13 um2. Script B then uses it much more often: in fir10 it goes from 237 to 403 instances, saving 1,187.7 um2, and in dsp8 from 27 to 40 instances, saving 102.6 um2. Script A never uses it.

Even so, the total area (taking the better script for each design) only moves from 86,102.6 to 86,100.7 um2, a change of 1.9 um2. The reason is that on those designs script A with the original library is still smaller than script B with the new cell, so the better result does not change. ABC only knows the areas written in the Liberty file. This means script B's result depends strongly on our area estimate, but the overall best result hardly depends on it at all.

part 4 step 9: worst delay

The worst delay of each design, from OpenSTA (results/worst_delay.csv, also included in both Table 5 files).

With the estimated cells

The biggest slowdown is cpu8 with script B, from 4.11 to 4.24 ns (3% slower). Several script B designs actually get faster: risc8 from 13.32 to 10.98 ns, arlet6502 from 8.65 to 6.87 ns, and dsp8 from 5.63 to 4.70 ns. This happens because one complex gate replaces a chain of several gates. Note that these delays use the placeholder timing values from section 6, not measured ones.

With the real cells

The real cells have measured timing, but since the netlists barely change, most delays stay close to the original. The biggest change is arlet6502 with script B, from 8.65 to 6.27 ns.

part 4 step 10: gate-level testbenches

Tutorial 4's testbenches run on the SPM and mul netlists (results/gate_level_testbenches.csv).

All 24 runs pass. They cover the original netlist, the netlist with estimated cells, the final netlist with estimated cells, and the final netlist with real cells. This includes the exhaustive testbench, which tries all 65,536 pairs of multiplier inputs.

part 5: discussion

How many 4-input classes are there under PO and under NPN, and why does the PO count matter for CMOS?

We found 362 PO classes and 91 NPN classes. In CMOS, reordering the inputs of a cell is just a change in wiring, and an inverted output is free because the pull-up and pull-down networks can simply be swapped. Inverting an input, however, needs a real inverter, which costs 2 transistors and usually one extra site. NPN treats that inverter as free. So the smaller NPN count makes it look as if one cell covers more functions than it really does.

How many candidates did each Table 3 label remove? (Examples are in part 4 step 2.)

Exists removed all 53 of its classes. Exists with an inverter removed 5 of its 28, and Contained removed 2 of its 30. The area exceptions turned out to matter: f07ff (0x07FF) is in our final set only because its estimated area was smaller than the HD cell a32oi.

Why is the saving different from one design to another?

The cells we kept are single-stage functions that look like parts of a carry chain. Script B uses ABC's &nf command, which matches cells by their logic function, so it can use them wherever multiply-accumulate or carry logic repeats: fir10 saves 2.10% and cpu8 1.75% with script B. Script A uses amap, which matches cells by the structure of the graph, and, as the handout warns, it never uses a cell that has the same input twice in its function. On cpu8, fir10, spm and mul, script A with the original library already gives the smallest result, so adding our cells does not improve the better of the two. Savings only remain where script B with the new cells beats script A without them: arlet6502 0.49%, risc8 0.26%, dsp8 0.23% and uart_apb 0.07%.

Why does the saving disappear with the real cells?

Our area model was fitted to HD's x1 cells, which were laid out by hand and packed very tightly. The generator's layouts share diffusion between transistors but need extra sites to leave room for routing, so the real cells came out 2 to 3 times larger than our estimate. At those sizes ABC rarely chooses them, and the 0.17% saving becomes 0.01%. If the cells could be made as dense as HD's, these functions would be worth adding. At 2 to 3 times HD's size, they are not. In the end, the measured result decides, not how often a function appears in the cuts.

Do risc8, pic16 and p16c5x, which all implement the same instruction set, benefit from the same new cells? (results/isa_compare.md)

No. When each design is ranked on its own, risc8 and pic16 have no candidate with a positive estimated saving, and p16c5x has one: 0x07FF, with an estimated saving of 72.63 um2, which is also our third final cell. Their most common New classes overlap much more: pic16 and p16c5x share 9 of their top 10, and risc8 shares 7 of its top 10 with each of them. So the same instruction set leads to similar logic, but the exact structure of each synthesized graph decides which cuts exist and which of them actually save area.

An example of a common function that did not save area, and why.

0x077F is the handout's own maj4n, the carry output of a carry-save adder. It appears often: script B used it 31 times in SPM alone. Yet SPM with script B got 1.33% larger with this cell (results/table5_estimated.md), and in the greedy step it saved 0.0 um2. Putting the carry into a single cell breaks up the majority and XOR logic that neighbouring gates were sharing, so ABC has to pay for that logic somewhere else. In the same way, 0xEEFA was ranked first, with an estimated saving of 36.7 um2, but it actually made the total 37.5 um2 larger. How often a function appears says nothing about whether the logic around it gets cheaper. Only mapping the design again shows that.

part 6: limitations

What our area model, Liberty file and the two scripts do not capture.

One row of Table 4 does not match

For the row "New, one stage, at most 3 transistors in series and at most 12 transistors" we count 6 classes, while the handout says 5. For each of our 6 we have an explicit 12-transistor circuit with no more than 3 transistors in series, and the tool re-checks it from its formula

(results/table4_selftest.txt). The 6 classes form 3 dual pairs (each is the other with the pull-up and pull-down networks swapped), so any rule that treats both networks the same way must give an even number. We report this difference and did not change our tool to hit the handout's number.

The area model

It is a straight line in the number of transistors, with a mean error of about 15% and a worst error of 3 sites. It also assumes HD's density, which the generator does not reach; its cells were 2 to 3 times larger.

The cost model

It gives an upper bound on the cost. It does not share inverters between stages and does not search every possible way to factor a function.

The ranking

It is a guess by design. The structure of the graph limits which cuts exist, and the top-ranked candidate turned out worse than adding no cell at all.

The greedy selection

Each candidate is tried only once, in ranked order. A candidate dropped early is never tried again after other cells are added.

The Liberty file

The estimated cells have only one fixed delay value per input. The real cells have measured timing tables, but only for the typical corner (TT): there are no slow or fast corners and no power data.

The scripts

amap ignores cells that use the same input twice, and &nf -R 1000 loosens the delay constraint. Just changing the library, without our cells being used, can move script A's area by about 1%.

The tools

Table 1 only reproduces with native Yosys 0.33. yowasp-yosys 0.33 and Yosys 0.70 give different areas. cellweaver needed its own tested builds of Magic and Netgen (container/Dockerfile).

part 7: contributions

Who did what. Every member agrees with this list.

Abdelrahman Rashed (Person B)

Checkpoint 1: AGENTS.md, truth tables and classes, the Liberty and AIGER file readers, the cut finder and its checks, the redundancy checker and its Table 4 self-test, and Table 1.

Checkpoint 2: the cell requests, the Liberty writer, the mapping, the equivalence checks, the gate-level testbenches, and Table 5.

Checkpoint 3: the interview demo, the OpenSTA delays, building the real cells with cellweaver, the Appendix A responses, the README and the reports.

Ziad Khairaldin (Person A)

Checkpoint 2: the cost model, the area model, and the candidate finder and ranking.

Checkpoint 3: the greedy final-set selection, the curve, the final Table 5, the 30% experiment, the instruction-set comparison, and the hook that plugs in the real cells.

appendix: tool versions

Tool	Version
Yosys and yosys-abc	0.33 (git sha1 2584903a060), from Ubuntu 24.04 apt, inside the course Docker image dd2-layout-tools
Python / numpy	3.12.3 / 1.26.4
HD Liberty file	sky130_fd_sc_hd__tt_025C_1v80.lib, open_pdks bdc9412b, sha256 e66aab4e...2c111baf
Icarus Verilog	13 (on the host)
Magic / Netgen / ngspice	in the course image: 8.3.684 / 1.5.133 / 47 with the BSIM4 fix; for cellweaver: Magic 4f53bb3 and Netgen 1.5.323 e1528a7
OpenSTA	3.1.0, built from source at f8e13cb
cellweaver	github.com/shalan/cellweaver e7fd28e, with the sky130_fd_pr models from the same open_pdks version